



Rapport de TER (Travail d'Étude et de Recherche)

Master 1 Informatique - Parcours SeCRéTS

Sécurité des Contenus, des Réseaux, des Télécommunications et des Systèmes

Université Paris-Saclay - Campus UVSQ (Versailles)

Détection efficace de propriétés statistiques de chiffrements symétriques

Présenté par :

Alexandre FRANÇOIS
Binet DUPRÉ
Bastien GUIBERT

Sous la direction de :

Jules BAUDRIN
Laboratoire de Mathématiques de Versailles (LMV) - UVSQ

Université Paris-Saclay
Année universitaire 2025-2026
22 avril 2026

Table des matières

1	Introduction	2
2	L'Algorithme naïf absolu : la recherche exhaustive	3
2.1	Principe théorique (version brute)	3
2.2	Complexités de l'algorithme brut	4
2.3	De la théorie à la pratique : Nos optimisations	4
2.4	Pseudo-code de notre implémentation	5
2.5	Limites matérielles et stratégies d'adaptation	5
3	L'Algorithme de recherche standard : une approche statistique	6
3.1	Principes et fondements statistiques : distinguer le signal du bruit	6
3.2	Pseudo-code de notre implémentation	7
3.3	Analyse de l'implémentation C++ et limites	8
4	L'Algorithme fondamental : l'exploitation des collisions	8
4.1	Le concept de différentielle de substitut	9
4.2	Le mécanisme du « quadruplet correct » (<i>right quartet</i>)	9
4.3	Gain de complexité et dimensionnement de l'échantillon M	11
4.4	Démonstration mathématique des seuils de détection et vérification	11
4.5	Implémentation, complexités et limite de la mémoire vive	13
5	L'algorithme sans mémoire (<i>memoryless</i>) : le prix du temps	15
6	L'algorithme de compromis temps/mémoire (tradeoff)	16
6.1	Principe théorique : l'approche probabiliste par lots (batching)	16
6.2	De la théorie à la pratique : implémentation par lots et accumulation globale . . .	17
7	Ouverture : La dépendance aux hypothèses	18
8	L'algorithme « pire cas » (<i>worst-case</i>) : s'affranchir des heuristiques	19
8.1	Paramétrage et seuils de validation	19
8.2	Complexités théoriques et implémentation	20
8.3	Limites pratiques : le cumul des contraintes	20
8.4	Bilan et synthèse comparative des algorithmes	21
9	Analyse comparative des performances sur SPN 16 bits	22
10	Les limites de l'approche naïve face à Speck 32/64 (3 tours)	22
11	Validation expérimentale et résolution matérielle (Speck 5 et 6 tours)	23
11.1	Reproduction des résultats sur 5 tours de Speck 32/64	23
11.2	Reproduction des résultats sur 6 tours de Speck 32/64	24
12	Conclusion générale et perspectives	25

1 Introduction

La sécurité des communications numériques modernes repose en grande partie sur la robustesse des primitives de chiffrement symétrique. Pour garantir la confidentialité des données, ces algorithmes doivent se comporter comme des permutations parfaitement aléatoires. Néanmoins, à l'exception de constructions théoriques impraticables comme le masque jetable, il est impossible de prouver mathématiquement la sécurité absolue des primitives symétriques modernes. Dès lors, leur évaluation continue par la cryptanalyse s'impose comme une nécessité. Le travail du cryptanalyste consiste alors à déceler des failles dans cette conception en identifiant, par exemple, des biais statistiques permettant de distinguer le chiffrement d'une fonction purement aléatoire. Parmi les techniques les plus puissantes figure la cryptanalyse différentielle [1], introduite par Biham et Shamir à la fin des années 1980. Cette méthode analyse la propagation d'une différence d'entrée spécifique à travers les tours du chiffrement dans l'espoir d'observer une différence de sortie avec une probabilité anormalement élevée.

Génériquement, la découverte de telles caractéristiques différentielles reposait sur l'évaluation systématique d'une pléthore de textes clairs. L'approche triviale consiste à construire la table de distribution des différences (DDT) complète. Elle exige une complexité en temps et en mémoire de l'ordre de 2^{2n} (où n est la taille du bloc). Si cette méthode est viable pour des chiffrements de taille réduite, elle se heurte à l'explosion combinatoire dès que l'on cible des tailles de blocs de 64 ou 128 bits. L'évaluation exhaustive devient alors physiquement impossible, car elle requiert des ressources qui dépassent les capacités des supercalculateurs mondiaux.

C'est dans ce contexte que s'inscrivent les algorithmes de recherche probabilistes basés sur les collisions. En exploitant des propriétés mathématiques telles que par l'utilisation d'une dérivée de substitution et le paradoxe des anniversaires, il est théoriquement possible de ramener cette complexité à l'ordre de $2^{n/2} \cdot p^{-1}$ (où p représente la probabilité de la différentielle recherchée)

Le présent Travail d'Étude et de Recherche (TER) porte sur cette avancée théorique. Notre objectif n'a pas été de nous limiter à une étude bibliographique, mais de développer un moteur d'analyse cryptographique hautes performances en C++ afin de confronter ces modèles mathématiques à la réalité de la programmation logicielle et matérielle.

Dans ce rapport, nous détaillerons le fonctionnement de l'algorithme fondamental par collisions [3] et l'importance de ses filtres probabilistes (loi de Poisson) avant d'exposer la contrainte critique du « mur de la RAM ». Pour surmonter cette impasse, nous étudierons les variantes de compromis temps/mémoire (*Tradeoff*) [3] ainsi que l'algorithme de « pire cas » [3]. Enfin, nous présenterons une analyse critique de nos résultats expérimentaux. Au travers de tests menés sur des architectures de 12 et 16 bits jusqu'à l'application sur le chiffrement Speck 32/64 [4], nous mettrons en évidence la hiérarchie de performances des différents algorithmes étudiés.

Notations et définitions préliminaires

Afin de faciliter la lecture des développements algorithmiques et mathématiques présentés dans ce rapport, nous fixons les notations suivantes :

- n : désigne la taille du bloc du chiffrement (en bits). L'espace des messages (textes clairs) est noté $\{0, 1\}^n$.
- F : représente la primitive cryptographique étudiée.
- x : désigne un message clair appartenant à $\{0, 1\}^n$.
- α : représente la *différence d'entrée* entre deux messages, définie par le OU exclusif (XOR) :

$$\alpha = x \oplus x'$$

- β : représente la *différence de sortie* observée après chiffrement, telle que :

$$\beta = F(x) \oplus F(x \oplus \alpha)$$

- p : désigne la probabilité d'une caractéristique différentielle (α, β) . Elle est définie par la probabilité que deux messages ayant une différence α produisent une différence de sortie β :

$$p = \Pr_x[F(x) \oplus F(x \oplus \alpha) = \beta]$$

- γ : désigne le *substitut*, une différence arbitraire non nulle utilisée pour sonder le chiffrement indépendamment de α .
- g_γ : représente la fonction dérivée de F dans la direction du substitut γ , définie par :

$$g_\gamma(x) = F(x) \oplus F(x \oplus \gamma)$$

- **DDT** : désigne la *Difference Distribution Table* (Table de Distribution des Différences), matrice de taille $2^n \times 2^n$ recensant le nombre d'occurrences de chaque couple (α, β) .
- **pDDT** : désigne la *partial Difference Distribution Table* (Table de Distribution des Différences partielle). Contrairement à la DDT complète, cette structure de données ne conserve en mémoire que les caractéristiques différentielles (α, β) dont la probabilité d'occurrence atteint ou dépasse un seuil de filtrage prédéfini.

2 L'Algorithme naïf absolu : la recherche exhaustive

La méthode la plus directe pour détecter des différentielles au sein d'un chiffrement symétrique consiste à évaluer de manière exhaustive toutes les paires possibles de textes clairs. Cet algorithme, que nous qualifierons de « naïf absolu », sert de point de référence théorique pour comprendre la mécanique de la cryptanalyse différentielle et construire la **Table de Distribution des Différences (DDT)**.

Table de Distribution des Différences

Pour une fonction de chiffrement $F : \{0, 1\}^n \rightarrow \{0, 1\}^n$, la DDT est une matrice de dimension $2^n \times 2^n$ dont chaque entrée (α, β) est définie par :

$$DDT[\alpha][\beta] = \#\{x \in \{0, 1\}^n \mid F(x) \oplus F(x \oplus \alpha) = \beta\}$$

Chaque entrée de la table représente le nombre d'occurrences où une différence d'entrée α produit une différence de sortie β . Cette table est symétrique par rapport à la parité (toutes les entrées sont paires) et sa première ligne/colonne ($DDT[0][0]$) est toujours égale à 2^n , correspondant à la différence triviale. La probabilité p d'une caractéristique différentielle est directement liée aux valeurs de cette table par la relation $p = \frac{DDT[\alpha][\beta]}{2^n}$.

2.1 Principe théorique (version brute)

Nous détaillons ici, l'algorithme dans sa version la plus simple, c'est-à-dire sans utilisation de filtre ou autre optimisation. Pour chaque différence d'entrée possible α (allant de 1 à $2^n - 1$, où n est la taille du bloc en bits), il parcourt l'intégralité de l'espace des messages x (soit 2^n messages). La boucle sur α commence délibérément à 1 et non à 0, car une différence $\alpha = 0$ implique des messages identiques ($x \oplus 0 = x$), conduisant toujours à une différence de sortie triviale $\beta = 0$, sans aucun intérêt cryptanalytique.

Pour chaque message x , la version brute calcule les chiffrés à la volée, évalue la différence de sortie $\beta = F(x) \oplus F(x \oplus \alpha)$, et incrémente la case correspondante dans une matrice globale (la DDT complète de taille $2^n \times 2^n$).

2.2 Complexités de l'algorithme brut

L'évaluation de cette approche naïve met en évidence des limites importantes :

- **Complexité en temps** : $\mathcal{O}(2^{2n})$. Il y a approximativement 2^n différences d'entrées α à tester. Pour chacune d'elles, l'algorithme parcourt les 2^n messages x afin d'évaluer les paires $(x, x \oplus \alpha)$. Puisque l'évaluation de chaque paire requiert deux appels à la fonction de chiffrement ($F(x)$ d'une part, et $F(x \oplus \alpha)$ d'autre part), le coût total de la recherche s'élève à environ 2×2^{2n} opérations. Il est important de noter que cette complexité est exponentielle par rapport au paramètre de taille du bloc n . Cette complexité exponentielle rend la méthode strictement inutilisable en temps raisonnable dès que $n > 40$.
- **Complexité en données** : $\mathcal{O}(2^n)$. L'algorithme nécessite le chiffrement de l'ensemble de l'espace des textes clairs possibles.
- **Complexité en espace (mémoire)** : $\mathcal{O}(2^{2n})$. Stocker la DDT complète requiert l'allocation d'une matrice de taille $2^n \times 2^n$ entiers. Cette contrainte sature instantanément la mémoire physique dès que l'on aborde des tailles de blocs standards (typiquement 64 ou 128 bits pour les algorithmes modernes comme l'AES). À titre d'exemple, la construction d'une DDT pour un simple bloc de 64 bits générerait 2^{128} entrées. Même en supposant qu'une entrée ne pèse qu'un seul octet, la table exigerait des milliards de milliards de téraoctets (2^{88} To) de mémoire vive, dépassant de loin les capacités actuels des supercalculateurs.

2.3 De la théorie à la pratique : Nos optimisations

Pour pouvoir exécuter cette recherche exhaustive efficacement, même sur des « chiffrements jouets » ($n \leq 16$), l'implémentation théorique brute a dû être entièrement repensée. Dès le début du projet, le choix du langage C++ s'est imposé pour nous offrir un contrôle strict sur la gestion de la mémoire et les performances d'exécution (l'intégralité de notre code source optimisé est disponible sur notre dépôt GitHub public : <https://github.com/Redak92/cryptanalyse-differentielle>, dans le fichier `src/analysis/DifferentialSearch.cpp`). Nous avons articulé notre approche autour de trois optimisations majeures.

Le pré-calcul des chiffrés (compromis temps/mémoire) : Pour réduire la coût lié aux appels répétés à la fonction de chiffrement, nous avons mis en place un compromis temps/mémoire. Au lieu de calculer les chiffrés à la volée en permanence, l'ensemble des 2^n messages possibles est évalué une seule fois au tout début de l'exécution. En effet, dans une implémentation naïve, l'évaluation de la paire $(x, x \oplus \alpha)$ exige de calculer la primitive deux fois par itération. Puisque l'espace des messages est intégralement parcouru pour chaque différence α , une même valeur $F(x)$ se retrouve recalculée exactement deux fois par boucle (une fois en tant que terme de gauche, et une fois en tant que terme translaté), soit près de 2^{n+1} fois sur l'ensemble de la recherche. Grâce à ce pré-calcul, les évaluations sont stockées dans une table des valeurs (*look-up table*) de la fonction, que nous noterons T . Le calcul de la différence de sortie β se résume alors à deux simples accès mémoire directs suivis d'une disjonction exclusive : $T[x] \oplus T[x \oplus \alpha]$.

Du stockage exhaustif au filtrage à la volée : Lors de l'implémentation, nous avons rapidement pris conscience qu'allouer et conserver en mémoire l'intégralité d'une DDT classique allait inévitablement saturer la RAM, l'immense majorité de ces 2^{2n} entrées étant strictement non significatives (le bruit statistique). Pour pallier ce problème, nous avons modifié la logique en introduisant un paramètre de seuil de probabilité pour ne générer qu'une DDT partielle (pDDT). Le programme agit ainsi comme un filtre dynamique : il alloue un unique tableau temporaire de compteurs de taille 2^n . À la fin de chaque itération sur la différence d'entrée α , seules les différentielles atteignant le seuil critique (les véritables vulnérabilités) sont extraites et sauvegardées. Le tableau est ensuite simplement réinitialisé pour l'itération suivante. Tout le bruit de fond est ainsi ignoré, ce qui évite le stockage massif d'une matrice en $\mathcal{O}(2^{2n})$ et réduit drastiquement l'empreinte mémoire de la structure finale.

La nécessité de la parallélisation : Malgré le pré-calcul et le filtrage, la boucle principale de l'algorithme naïf reste particulièrement lourde. Face à la quantité d'évaluations requises, une exécution purement séquentielle constituait une limite temporelle majeure. Le passage au calcul parallèle s'est donc avéré indispensable pour rendre l'algorithme praticable. Grâce à l'API OpenMP, nous avons pu distribuer l'itération principale (le parcours des différences α) sur les différents cœurs du processeur. Afin d'éviter tout conflit d'accès concurrent, le code a été structuré pour que chaque fil d'exécution (*thread*) alloue dynamiquement son propre sous-tableau de compteurs, divisant ainsi drastiquement le temps d'exécution global.

2.4 Pseudo-code de notre implémentation

Le pseudo-code suivant résume l'algorithme tel qu'il a été implémenté et optimisé dans notre code source.

Algorithm 1 Recherche Différentielle Naïve Optimisée (Génération de pDDT)

Entrées : F (fonction de chiffrement), n (taille du bloc), p (seuil de probabilité)

Sortie : Liste des différentielles (α, β) ayant une probabilité $\geq p$

```

1:  $limit \leftarrow 2^n$ 
2:  $minCount \leftarrow p \times limit$ 
3: Initialiser un tableau  $f_{table}$  de taille  $limit$ 
4: // Remplissage de la  $f_{table}$  (Pré-calculs)
5: for  $x \leftarrow 0$  to  $limit - 1$  do
6:    $f_{table}[x] \leftarrow F(x)$ 
7: end for
8: // Boucle parallélisée (OpenMP)
9: for  $\alpha \leftarrow 1$  to  $limit - 1$  do
10:   Initialiser un tableau  $counts$  de taille  $limit$  à 0
11:   Initialiser une liste vide  $candidates$ 
12:   for  $x \leftarrow 0$  to  $limit - 1$  do
13:      $\beta \leftarrow f_{table}[x] \oplus f_{table}[x \oplus \alpha]$ 
14:      $counts[\beta] \leftarrow counts[\beta] + 1$  // On retient  $\beta$  au moment exact où il franchit le seuil
15:     if  $counts[\beta] == minCount + 1$  then
16:       Ajouter  $\beta$  à  $candidates$ 
17:     end if
18:   end for
19:   // Sauvegarde finale (boucle uniquement sur les entrées valides)
20:   for chaque  $\beta$  dans  $candidates$  do
21:     Sauvegarder  $(\alpha, \beta, counts[\beta])$ 
22:   end for
23: end for

```

2.5 Limites matérielles et stratégies d'adaptation

Bien que très efficace sur des petites tailles de bloc, cette implémentation s'est directement heurtée au « mur de la RAM » lors de notre passage à l'échelle. Lors de nos premières tentatives d'exécution de l'algorithme naïf sur Speck 32/64 ($n = 32$), nous avons constaté une saturation immédiate de la mémoire vive (16 Go) de nos stations de travail. Le simple fait de vouloir allouer la table des valeurs de F nécessitait théoriquement environ 34,36 Go de RAM. Ce chiffre découle directement de la taille de l'espace des messages : le stockage de 2^{32} valeurs, chacune codée sur un entier standard de 8 octets (64 bits), donne très exactement $\frac{2^{32} \times 8}{1024^3} \approx 34,36$ Go.

Ce blocage systématique, se traduisant par un plantage du programme par dépassement de mémoire (*Out-Of-Memory*), nous a forcés à abandonner l'idée d'un pré-calcul intégral. Lorsque la

taille du bloc excède les capacités physiques de la machine, le programme doit impérativement être adapté pour préserver la mémoire.

Plusieurs stratégies ont dû être envisagées. La solution la plus radicale consiste à supprimer totalement la phase de pré-calcul. En ne stockant plus aucun résultat à l'avance, l'algorithme se voit contraint d'évaluer la fonction de chiffrement à la volée pour chaque itération. Si la consommation spatiale redevient ainsi minime, le temps d'exécution s'allonge de façon critique. En effet, bien que la complexité asymptotique des boucles demeure inchangée (toujours de l'ordre de $\mathcal{O}(2^{2n})$ itérations), le remplacement d'un accès direct en mémoire par un appel cryptographique complet démultiplie le coût unitaire de chaque opération.

Une autre approche pour réduire l'empreinte mémoire repose sur la désactivation de la parallélisation. En forçant un retour à un mode d'exécution strictement séquentiel, le programme évite l'allocation simultanée de multiples et volumineux tableaux de compteurs par les différents fils d'exécution du processeur. Enfin, une dernière optimisation repose sur le nettoyage de la mémoire à la volée : plutôt que de tout conserver dans une structure globale, l'algorithme affiche immédiatement sur la sortie standard les paires trouvées pour un α donné, purgeant sa mémoire avant de passer à la différence suivante.

Cependant, il est important de souligner que si ces adaptations logicielles permettent d'éviter la saturation de la machine, elles ne résolvent en rien la limitation algorithmique fondamentale. Contourner la contrainte spatiale ne fait que déplacer le problème vers la complexité temporelle : même avec une empreinte mémoire parfaitement maîtrisée, l'exécution des 2^{64} itérations requises pour analyser Speck 32/64 exigerait des semaines de calcul sur un ordinateur standard. Ces observations démontrent que la méthode exhaustive, même optimisée, est incapable de passer à l'échelle, ce qui justifie la nécessité de se tourner vers des approches de détection probabilistes.

3 L'Algorithme de recherche standard : une approche statistique

Bien que l'algorithme exhaustif soit utile pour des paramètres très réduits, sa complexité en $\mathcal{O}(2^{2n})$ le rend strictement impraticable dès que la taille du bloc atteint 32 bits. Pour pallier cette limitation, une approche naturelle consiste à transformer la recherche déterministe en un problème d'échantillonnage statistique. Cet algorithme est décrit par Dinur et al. [3] comme l'adaptation classique de la construction d'une DDT pour détecter des différentielles de probabilité supérieure à un seuil p .

3.1 Principes et fondements statistiques : distinguer le signal du bruit

Plutôt que de parcourir l'intégralité de l'espace des messages, l'algorithme statistique standard, tel que détaillé à la page 7 de l'article de Dinur *et al.* [3], privilégie une stratégie par échantillonnage. Pour chaque différence d'entrée α , nous testons un nombre fixe k de paires aléatoires. Cette approche transforme la recherche en un défi statistique où l'objectif est de faire émerger un « signal » (une différentielle réelle) d'un « bruit » de fond aléatoire.

Sous l'hypothèse d'indépendance des paires évaluées, le nombre de succès X , représentant le nombre de paires dont la différence d'entrée est α et la différence de sortie est β , peut être modélisé par une loi binomiale $\mathcal{B}(k, p)$. Cependant, face à un nombre de tirages k élevé et une probabilité de succès p très faible, nous basculons dans le domaine de la **loi des événements rares**. La distribution de X peut alors être approximée par une **loi de Poisson** $\mathcal{P}(\lambda)$. Dans ce modèle, le paramètre λ représente l'espérance mathématique du nombre de succès, soit $\lambda = k \times p$. La probabilité d'obtenir exactement x succès est donnée par la formule :

$$P(X = x) = \frac{\lambda^x e^{-\lambda}}{x!}$$

Afin d'assurer une forte probabilité de détection pour toute différentielle significative (et ainsi minimiser le risque de faux négatif), les auteurs fixent $k = 4/p$ (soit $\lambda = 4$). La probabilité de succès se calcule alors par le complémentaire :

$$P(X \geq 2) = 1 - (P(X = 0) + P(X = 1)) = 1 - \left(\frac{\lambda^0 e^{-\lambda}}{0!} + \frac{\lambda^1 e^{-\lambda}}{1!} \right) = 1 - e^{-\lambda}(1 + \lambda)$$

Pour $\lambda = 4$, on obtient $1 - 5e^{-4} \approx 0,9085$, ce qui permet d'atteindre le seuil de confiance de 90 % visé par la littérature.

Le choix du seuil de validation $count \geq 2$ est crucial pour filtrer les faux positifs. En effet, si l'on modélise le chiffrement par une permutation aléatoire, la probabilité qu'une paire de messages présente la différence de sortie β par pur hasard est extrêmement faible ($p_{rand} = \frac{1}{2^n - 1}$). À titre d'exemple, pour l'analyse de Speck 64 ($n = 64$) ciblant une probabilité $p = 2^{-13}$, l'espérance du nombre de paires accidentelles, notée $\lambda_{rand} = k \times p_{rand}$, n'est que d'environ 2^{-49} .

Pour évaluer la probabilité qu'un tel bruit génère un faux positif, nous utilisons le **développement limité de Taylor** de l'exponentielle au voisinage de zéro.

Ce développement est ici indispensable car λ est extrêmement petit. En effet, par développement de Taylor de $e^{-\lambda}$ au voisinage de 0, on obtient $e^{-\lambda} = 1 - \lambda + \frac{\lambda^2}{2} + o(\lambda^2)$, ce qui permet de simplifier l'exponentielle et d'évaluer des probabilités extrêmement faibles. En injectant cette approximation dans notre formule factorisée, le calcul devient :

$$P(X \geq 2) \approx 1 - \left(1 - \lambda + \frac{\lambda^2}{2} \right) (1 + \lambda) = 1 - \left(1 + \lambda - \lambda - \lambda^2 + \frac{\lambda^2}{2} + \frac{\lambda^3}{2} \right)$$

En négligeant le terme en λ^3 (d'ordre supérieur et donc totalement insignifiant face à λ^2 quand $\lambda \rightarrow 0$), on aboutit à :

$$P(X \geq 2) \approx 1 - \left(1 - \frac{\lambda^2}{2} \right) = \frac{\lambda^2}{2}$$

Dans notre exemple, cette probabilité de faux signal chute à environ 2^{-99} , ce qui rend l'apparition d'une telle paire accidentelle statistiquement peu probable et prouve que les faux positifs sont très rares.

3.2 Pseudo-code de notre implémentation

Le pseudo-code suivant résume l'algorithme tel qu'il a été implémenté et optimisé dans notre code source.

Algorithm 2 Recherche Statistique Standard par Échantillonnage

Entrées : F (fonction de chiffrement), n (taille du bloc), p (seuil de probabilité)

Sortie : Liste des candidats différentiels $(\alpha, \beta, \text{occurrences})$

```
1:  $limit \leftarrow 2^n$ 
2:  $k \leftarrow 4/p$  ▷ Nombre de paires aléatoires à évaluer par  $\alpha$ 
3:  $expectedHits \leftarrow k \times p$ 
4:  $minThreshold \leftarrow \max(3, \frac{expectedHits}{2})$  ▷ Calcul dynamique du seuil de filtrage
5: Initialiser une liste globale  $resultats$ 
6: // Boucle parallélisée (répartition sur les cœurs du processeur)
7: for  $\alpha \leftarrow 1$  to  $limit - 1$  do
8:   Initialiser une table de hachage locale  $counts$  (vide)
9:   Initialiser une liste locale  $candidats$ 
10:  // Phase d'échantillonnage aléatoire
11:  for  $i \leftarrow 1$  to  $k$  do
12:     $x \leftarrow \text{Générer\_Bloc\_Aléatoire}(n)$ 
13:     $y \leftarrow x \oplus \alpha$ 
14:     $\beta \leftarrow F(x) \oplus F(y)$ 
15:     $counts[\beta] \leftarrow counts[\beta] + 1$ 
16:    // CRITÈRE DE FILTRAGE : Ajout direct au franchissement du seuil
17:    if  $counts[\beta] == minThreshold + 1$  then
18:      Ajouter  $\beta$  à  $candidats$ 
19:    end if
20:  end for
21:  // Sauvegarde finale (uniquement sur les entrées valides de ce sous-lot)
22:  for chaque  $\beta$  dans  $candidats$  do
23:    Ajouter  $(\alpha, \beta, counts[\beta])$  à  $resultats$ 
24:  end for
25: end for
26: Retourner  $resultats$ 
```

3.3 Analyse de l'implémentation C++ et limites

Notre implémentation, disponible dans le fichier `src/analysis/DifferentialSearch.cpp`, parallélise l'itération sur les différences d'entrée α en répartissant la charge sur plusieurs fils d'exécution. Contrairement à l'algorithme exhaustif, l'utilisation d'une table de hachage dynamique permet de ne conserver que les différences de sortie effectivement rencontrées, optimisant ainsi considérablement l'empreinte mémoire.

Toutefois, comme le met en évidence la boucle principale du pseudo-code itérant sur l'ensemble des différences d'entrée α , cette approche reste limitée par le facteur 2^n . Pour un chiffrement comme **Speck 64/128**, l'algorithme doit itérer, à minima, 2^{64} fois, ce qui est prohibitif sur un ordinateur personnel.

C'est cette barrière qui justifie l'introduction de l'algorithme probabiliste présenté dans l'article. En s'appuyant sur le **paradoxe des anniversaires**, ce dernier permet de convertir la recherche de différentielles en une recherche de collisions, abaissant la complexité à l'ordre de $2^{n/2} \cdot p^{-1}$.

4 L'Algorithme fondamental : l'exploitation des collisions

Pour franchir la barrière du 2^n , cet algorithme délaisse l'approche par différence d'entrée fixe pour s'appuyer sur la **différentielle de substitut**. L'objectif est de détecter toutes les différentielles $\alpha \rightarrow \beta$ de probabilité p avec une complexité temporelle et spatiale réduite à l'ordre de $\tilde{O}(2^{n/2} \cdot p^{-1})$.

4.1 Le concept de différentielle de substitut

La nouvelle approche du papier consiste à choisir une différence γ arbitraire et non nulle, appelée **substitut**, totalement indépendante de la différence α recherchée. On définit la fonction g_γ comme la dérivée de F dans la direction du substitut :

$$g_\gamma(x) = F(x) \oplus F(x \oplus \gamma)$$

Pour appréhender ce gain de complexité, il convient de formaliser la distinction mathématique entre les deux approches :

- **L'approche standard (Recherche itérative)** : La différence d'entrée α est fixée *a priori*. L'algorithme cherche des paires satisfaisant :

$$F(x) \oplus F(x \oplus \alpha) = \beta$$

Cette condition impose de boucler séquentiellement sur l'intégralité de l'espace des différences d'entrée, soit $\forall \alpha \in \{0, 1\}^n \setminus \{0\}$, entraînant une complexité en $\mathcal{O}(2^n)$.

- **La différentielle de substitut (Recherche de collisions)** : L'algorithme utilise $g_\gamma(x) = F(x) \oplus F(x \oplus \gamma)$ comme signature. Au lieu de fixer α , on cherche simplement une collision entre deux messages x et x' :

$$g_\gamma(x) = g_\gamma(x')$$

En développant cette égalité, on obtient :

$$F(x) \oplus F(x \oplus \gamma) = F(x') \oplus F(x' \oplus \gamma)$$

Si l'on pose $\alpha = x \oplus x'$, la différence d'entrée α se révèle naturellement d'elle-même comme une conséquence de la collision.

Grâce à cette formulation, il n'est plus nécessaire d'itérer sur chaque α . Le problème se réduit à une simple recherche de collisions globales sur g_γ , ce qui permet d'exploiter le paradoxe des anniversaires et de faire chuter la complexité temporelle à l'ordre de $\tilde{\mathcal{O}}(2^{n/2})$.

4.2 Le mécanisme du « quadruplet correct » (*right quartet*)

Le succès de l'algorithme repose sur une structure géométrique reliant quatre messages, formant ce que les auteurs appellent un quadruplet $\{x, x \oplus \alpha, x \oplus \gamma, x \oplus \alpha \oplus \gamma\}$. La propagation des différences au sein de ce quadruplet, à travers la fonction de chiffrement, est illustrée par le schéma de la Figure 1.

Définition 1 (Quadruplet correct). *Un tel quadruplet est dit **correct** pour la différentielle $\alpha \rightarrow \beta$ si le couple initial $(x, x \oplus \alpha)$ et le couple translaté par le substitut $(x \oplus \gamma, x \oplus \gamma \oplus \alpha)$ satisfont simultanément cette différentielle. Autrement dit, l'on doit observer à la fois $F(x) \oplus F(x \oplus \alpha) = \beta$ et $F(x \oplus \gamma) \oplus F(x \oplus \gamma \oplus \alpha) = \beta$.*

Comme le montre la Figure 1, la fonction g_γ permet d'annuler la valeur de β par le biais de cette double différenciation. En calculant la différence entre deux sorties de g_γ espacées de α , nous obtenons :

$$\begin{aligned} g_\gamma(x) \oplus g_\gamma(x \oplus \alpha) &= [F(x) \oplus F(x \oplus \gamma)] \oplus [F(x \oplus \alpha) \oplus F(x \oplus \alpha \oplus \gamma)] \\ &= [F(x) \oplus F(x \oplus \alpha)] \oplus [F(x \oplus \gamma) \oplus F(x \oplus \alpha \oplus \gamma)] \\ &= \beta \oplus \beta = 0 \end{aligned}$$

Cette égalité fondamentale implique que $g_\gamma(x) = g_\gamma(x \oplus \alpha)$, créant ainsi une **collision** directe sur les sorties de la fonction. Le point crucial de cette méthode est que la collision se produit **quelle que soit la valeur de β** . L'algorithme détecte donc la différence d'entrée α simplement en cherchant des doublons dans les sorties de g_γ , et ne calcule la différence de sortie β qu'a posteriori via l'opération $\beta = F(x) \oplus F(x \oplus \alpha)$.

Cette dérivation nous permet de poser le résultat suivant, qui constitue le cœur de l'efficacité de l'algorithme :

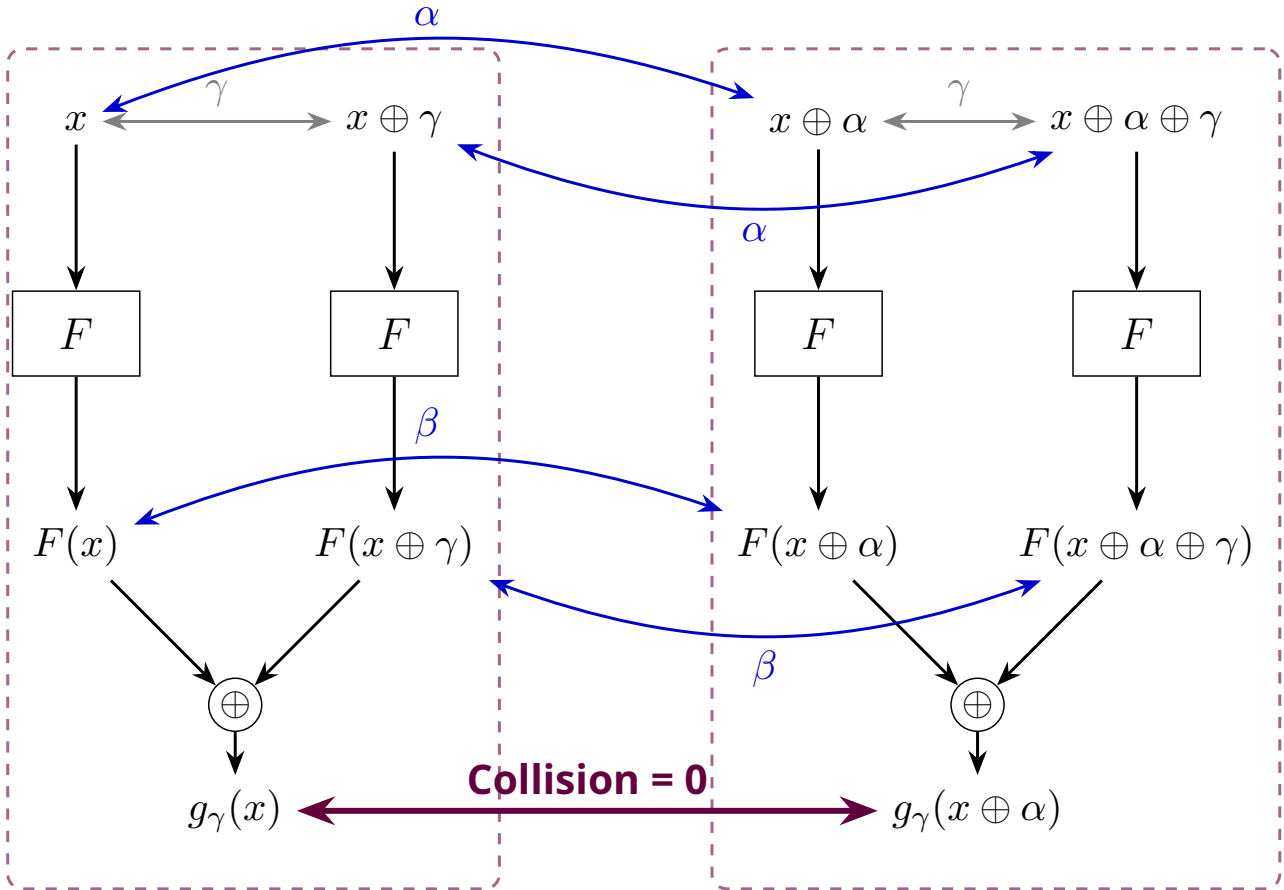


Figure 1 – Mécanisme du quadruplet correct et de la double différenciation. En posant $g_\gamma(x) = F(x) \oplus F(x \oplus \gamma)$, le fait que les couples $(x, x \oplus \alpha)$ et $(x \oplus \gamma, x \oplus \alpha \oplus \gamma)$ soient des paires correctes pour la différentielle $\alpha \rightarrow \beta$ (flèches bleues) entraîne mathématiquement une collision $g_\gamma(x) \oplus g_\gamma(x \oplus \alpha) = 0$, et ce quelle que soit la valeur de β .

Propriété 1 (Indépendance de la collision). *Soit un quadruplet $\{x, x \oplus \alpha, x \oplus \gamma, x \oplus \alpha \oplus \gamma\}$. Si ce quadruplet est correct pour une différentielle $\alpha \rightarrow \beta$, alors il existe nécessairement une collision pour la fonction g_γ :*

$$g_\gamma(x) = g_\gamma(x \oplus \alpha)$$

Cette égalité est indépendante de la valeur de β , ce qui permet de détecter la direction α sans connaissance préalable de la différence de sortie.

4.3 Gain de complexité et dimensionnement de l'échantillon M

Maintenant que nous avons établi qu'un quadruplet correct engendre nécessairement une collision dans g_γ (bien que la réciproque soit fautive en raison du bruit généré par des collisions purement aléatoires), la question centrale de l'algorithme devient : combien de messages aléatoires, que nous noterons M , devons-nous générer pour extraire ce signal avec certitude ?

C'est ici qu'intervient le **paradoxe des anniversaires**. Dans l'approche standard, une paire de messages a une probabilité p de satisfaire la différentielle. Dans cette nouvelle approche, sous **l'hypothèse d'indépendance** (postulant que la probabilité que le couple translaté satisfasse la propriété est indépendante de celle du couple initial), un message aléatoire x a une probabilité p^2 d'appartenir à un quadruplet correct.

Pour garantir la détection d'une différentielle, il est donc nécessaire de collecter un échantillon de taille M suffisamment large. Les auteurs fixent l'objectif d'obtenir une espérance de $E = n/2$ collisions engendrées par la bonne différentielle. Ce paramètre $n/2$ n'est pas choisi arbitrairement : il permet (via une modélisation par la loi de Poisson) de fixer un seuil de détection à $n/4$. Ce dimensionnement lié à la taille du bloc garantit un taux de réussite supérieur à 99 % pour les différentielles réelles, tout en rendant la probabilité d'accepter un faux positif négligeable (strictement inférieure à 2^{-n}).

Avec un ensemble de M messages, il est possible de former approximativement $\frac{M^2}{2}$ paires distinctes (x_i, x_j) . Pour une paire donnée, la probabilité que la différence $x_i \oplus x_j$ corresponde exactement à la valeur α recherchée est de 2^{-n} . De plus, sous l'hypothèse d'indépendance, la probabilité que les deux couples formés par ce quadruplet satisfassent simultanément la différentielle est égale à p^2 . Par conséquent, l'espérance du nombre de collisions suggérant la bonne différentielle s'exprime comme le produit de ces facteurs :

$$E[\text{Collisions}] = \frac{M^2}{2} \cdot \frac{1}{2^n} \cdot p^2$$

Afin d'assurer un signal statistiquement robuste, cette espérance est fixée à $n/2$. En résolvant cette équation pour M , on obtient le dimensionnement optimal :

$$\frac{M^2 \cdot p^2}{2 \cdot 2^n} = \frac{n}{2} \implies M^2 = \frac{n \cdot 2^n}{p^2} \implies M = \sqrt{n} \cdot 2^{n/2} \cdot p^{-1}$$

Ce résultat démontre mathématiquement comment le passage d'une recherche exhaustive à une recherche de collisions globales permet de réduire la complexité temporelle globale d'un facteur $2^{n/2}$.

4.4 Démonstration mathématique des seuils de détection et vérification

Une fois l'échantillon M généré, l'algorithme s'appuie sur trois seuils statistiques précis pour séparer le signal du bruit : $n/4$ pour la détection, puis n/p et $n/2$ pour la vérification. Ces valeurs ne sont pas empiriques, elles découlent d'une analyse rigoureuse basée sur la loi de Poisson.

Phase de détection : le seuil $n/4$ Pour une vraie différentielle de probabilité p , l'espérance du nombre de collisions (quadruplets corrects) parmi les M messages évalués est $\lambda = n/2$. Fondamentalement, la détection de chaque quadruplet correspond à une épreuve de Bernoulli indépendante, ce qui signifie que le nombre total de collisions observées, que nous noterons X , suit rigoureusement une **loi binomiale**. Cependant, face au nombre gigantesque de tirages (l'espace des messages) et à la rareté d'un succès (la probabilité p^2 étant infime), cette distribution binomiale s'approxime par une **loi de Poisson** de paramètre $\lambda = n/2$. On note ainsi :

$$X \sim \mathcal{P}\left(\frac{n}{2}\right)$$

L'algorithme configure son filtre pour ne conserver que les candidats (α, β) ayant généré au moins $n/4$ collisions. Pour prouver la robustesse de ce choix, il faut évaluer son comportement face au signal légitime, c'est-à-dire la validation des vrais positifs. La probabilité qu'une vraie différentielle soit correctement détectée correspond à $P(X \geq n/4)$. Puisque le seuil d'acceptation ($n/4$) est fixé exactement à la moitié de l'espérance théorique ($n/2$), la probabilité de rejeter à tort un candidat valide correspond à $P(X < n/4)$. Mathématiquement, cette probabilité d'échec s'obtient par le calcul direct de la fonction de répartition de la loi de Poisson :

$$P\left(X < \frac{n}{4}\right) = \sum_{k=0}^{\frac{n}{4}-1} e^{-\frac{n}{2}} \frac{\left(\frac{n}{2}\right)^k}{k!}$$

L'évaluation exacte de cette somme finie montre que l'erreur s'effondre très rapidement à mesure que la taille du bloc n augmente. Par exemple, en appliquant ce calcul pour $n = 64$ (soit une espérance $\lambda = 32$ et un seuil d'acceptation de 16), l'erreur devient négligeable et le taux de réussite (vrais positifs) atteint 99,9 %.

À l'inverse, l'algorithme doit être capable de rejeter le bruit statistique (les faux positifs). Considérons une fausse différentielle, définie par Dinur *et al.* [3] comme une propriété dont la probabilité est au plus $p/10$. Étant donné que la probabilité d'obtenir un quadruplet correct évolue de façon quadratique par rapport à la probabilité de la différentielle, l'espérance des collisions pour cette fausse piste s'effondre drastiquement :

$$\lambda_{false} = \frac{n}{2} \cdot \left(\frac{p/10}{p}\right)^2 = \frac{n}{2} \cdot \frac{1}{100} = \frac{n}{200}$$

La probabilité qu'une telle anomalie statistique franchisse le filtre de détection correspond à $P(Y \geq n/4)$, où $Y \sim \mathcal{P}(n/200)$. Atteindre le seuil de $n/4$ signifierait observer 50 fois l'espérance théorique ($n/200$). En évaluant la probabilité cumulée sur cette loi de Poisson pour une telle déviation, on démontre que le risque est majoré par 2^{-n} . Ainsi, la probabilité qu'une fausse piste soit validée par hasard s'avère négligeable, ce qui garantit un taux extrêmement faible de faux positifs parmi les candidats transmis à la phase de vérification.

Cette mécanique statistique mathématique est illustrée visuellement par la Figure 2. On y observe la séparation nette entre les deux distributions, validant la pertinence du seuil choisi pour isoler le signal du bruit statistique.

Phase de vérification : les seuils n/p et $n/2$ Bien que le filtre de détection élimine l'immense majorité du bruit, l'algorithme sécurise définitivement le résultat en testant les candidats survivants. Pour ce faire, il génère un nouvel échantillon de test, totalement indépendant du premier, composé de $N_{verify} = n/p$ paires aléatoires présentant la différence d'entrée α suspectée.

Si le candidat est bel et bien une vraie différentielle de probabilité p , chaque paire testée constitue une nouvelle épreuve de Bernoulli avec une probabilité de succès p . Le nombre de succès Z (c'est-à-dire les cas où la différence de sortie est effectivement β) suit alors une loi binomiale qui, tout comme dans la phase de détection, s'approxime par une loi de Poisson. L'espérance de cette distribution est :

$$E[Z] = \lambda_{verify} = N_{verify} \times p = \left(\frac{n}{p}\right) \times p = n$$

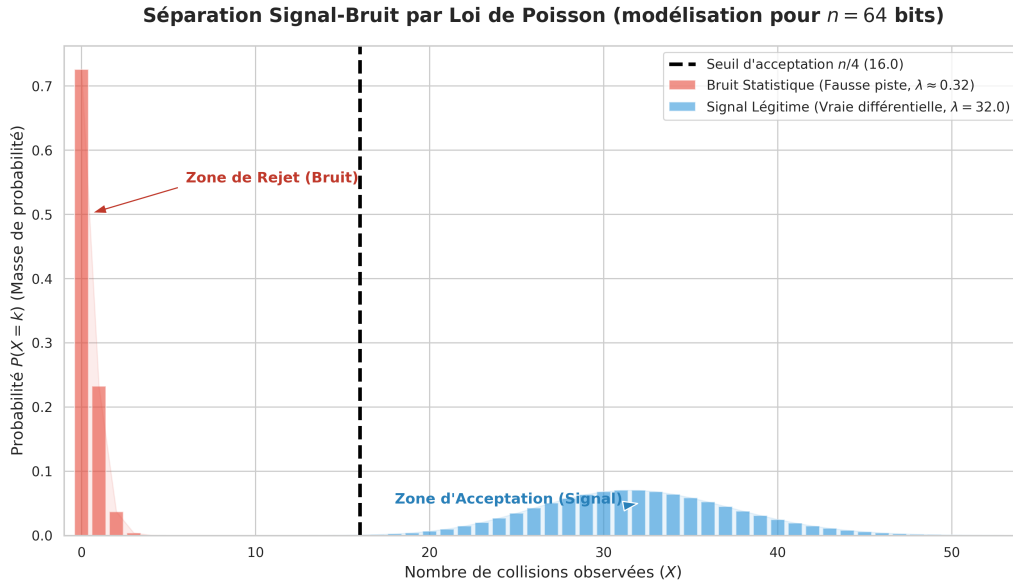


Figure 2 – Visualisation statistique de la séparation signal-bruit via le seuil $n/4$ (modélisation pour $n = 64$). La distribution légitime (droite) est parfaitement distincte du bruit (gauche), justifiant l'efficacité du filtrage mathématique.

L'algorithme valide définitivement la différentielle si le compteur de succès dépasse strictement le seuil de $n/2$. En plaçant à nouveau ce seuil d'acceptation à la moitié de l'espérance théorique, la logique mathématique reste identique à celle de la détection : la probabilité qu'une vraie différentielle échoue à ce stade (c'est-à-dire $P(Z \leq n/2)$) est négligeable et s'effondre avec la croissance de n .

À l'inverse, l'algorithme doit éviter les fausses différentielles (de probabilité $p/10$ ou moins) qui auraient pu franchir la première étape par pure malchance statistique. Pour un tel candidat, l'espérance lors de cette phase de vérification s'écroule :

$$\lambda_{false_verify} = N_{verify} \times \frac{p}{10} = \left(\frac{n}{p}\right) \times \frac{p}{10} = \frac{n}{10}$$

Pour qu'un tel faux positif soit validé, il faudrait qu'il produise plus de $n/2$ succès, soit observer au moins 5 fois son espérance théorique. Les propriétés de la loi de Poisson confirment que la probabilité d'une telle déviation est tout aussi négligeable que dans la phase de détection. Cette double barrière statistique garantit la fiabilité absolue des propriétés remontées par l'algorithme.

4.5 Implémentation, complexités et limite de la mémoire vive

Le pseudo-code détaillé de cet algorithme fondamental est fourni à la page 9 de l'article. Nous l'avons intégralement implémenté dans notre programme sous la fonction `runFundamentalAlgorithm`, dont le code source C++ est librement accessible sur notre dépôt GitHub public, au sein du fichier `src/analysis/DifferentialSearch.cpp`.

Grâce à cette approche exploitant la différentielle de substitut, la complexité temporelle de la recherche est drastiquement abaissée à $\mathcal{O}(n \cdot 2^{n/2} \cdot p^{-1})$. Tout au long du développement, nous avons dû faire des choix pour maximiser la vitesse d'exécution de cette implémentation. L'une de nos optimisations principales concerne le stockage et l'indexation des candidats différentiels : nous avons choisi de représenter les paires (α, β) d'entiers de 32 bits sous la forme d'un unique entier de 64 bits défini par $(\alpha \ll 32) \oplus \beta$.

Cependant, cette optimisation matérielle induit une limite structurelle stricte : puisque l'entier global fait 64 bits, il ne peut contenir que deux valeurs de 32 bits maximum. Ce choix de conception est un arbitrage que nous avons volontairement privilégié pour garantir des temps

de calcul rapides, compatibles avec la durée du TER. Si cette limite technique nous empêche d'analyser directement des standards de 128 bits comme l'AES, elle nous a néanmoins permis de valider l'intégralité de nos chaînes de détection algorithmiques sur des variantes réduites de Speck en quelques minutes. Pour adapter notre programme à des chiffrements de grande taille ($n = 64$ ou $n = 128$ bits), il faudrait abandonner cette concaténation au profit d'entiers beaucoup plus larges (de 128 ou 256 bits) ou revenir à l'utilisation de paires structurées.

Toutefois, même avec cette optimisation de stockage, la version fondamentale se heurte à une contrainte matérielle majeure : l'empreinte mémoire. Le maintien en mémoire d'une table de hachage globale pour stocker les M évaluations devient extrêmement prohibitif à mesure que la taille du bloc n augmente ou que le seuil de probabilité p diminue.

Pour illustrer théoriquement ce « mur de la RAM », calculons l'espace mémoire brut requis pour analyser un chiffrement aux paramètres pourtant modestes, tel que Speck 32/64 ($n = 32$), avec un seuil de probabilité $p = 2^{-13}$. Le nombre de messages M à évaluer et à stocker en phase de détection s'élève à :

$$M = \sqrt{n} \cdot 2^{n/2} \cdot p^{-1} = \sqrt{32} \cdot 2^{16} \cdot (2^{-13})^{-1} = \sqrt{32} \cdot 2^{29} = 2^{31,5} \text{ messages}$$

Sur le plan purement théorique, stocker chaque évaluation nécessite a minima de conserver le message d'entrée x et son résultat $g_\gamma(x)$ pour pouvoir identifier les collisions et en déduire α . Pour un bloc de 32 bits, cela représente 64 bits de données (soit 8 octets) par message. L'allocation de mémoire théorique brute, sans même compter le poids de la structure de données sous-jacente, s'élève donc à environ : $2^{31,5} \cdot 2^3 = 2^{34,5}$ octets, soit environ 24 Go.

Ainsi, même pour analyser un chiffrement « jouet » de 32 bits, l'algorithme exige théoriquement un minimum absolu de 24 Go de mémoire vive. À titre de comparaison, passer à un bloc standard de 64 bits avec la même probabilité ferait exploser ce besoin brut à plusieurs centaines de Téraoctets.

Pour visualiser concrètement cette impasse matérielle, nous avons modélisé la croissance de cette complexité spatiale en fonction de la taille du bloc (Figure 3).

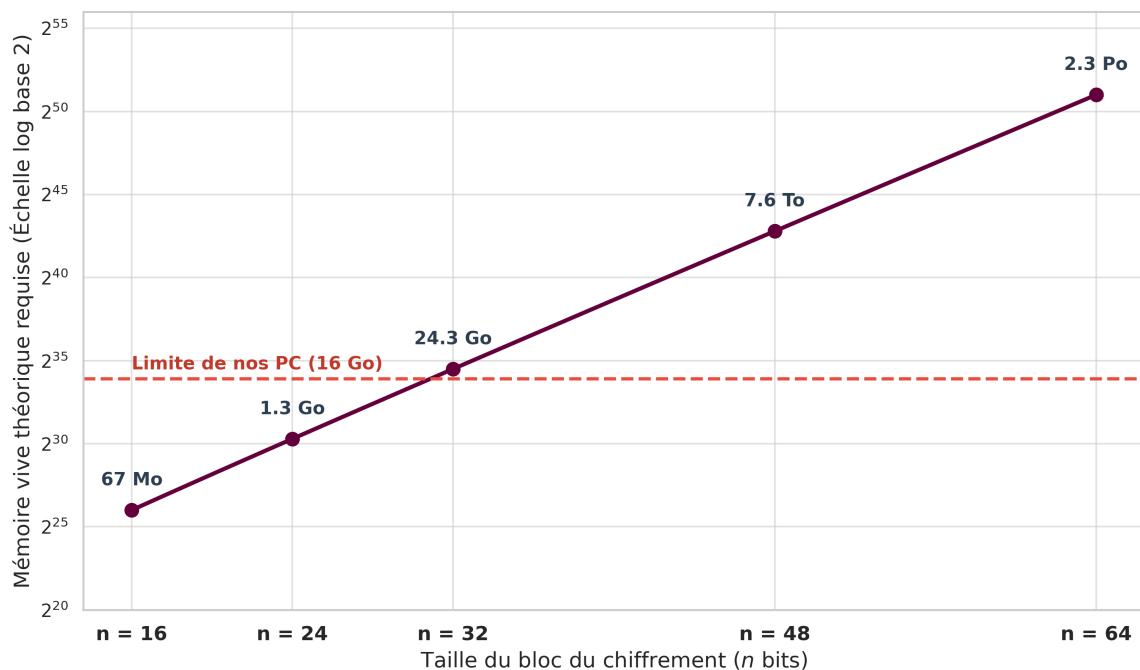


Figure 3 – L'explosion combinatoire de la mémoire : le « mur de la RAM ». Dès $n = 32$, l'algorithme fondamental pur dépasse les capacités d'un ordinateur personnel standard. Pour $n = 64$, il outrepassé les capacités des plus grands supercalculateurs mondiaux.

L'échelle logarithmique de ce graphique met en évidence la nature exponentielle de la fonction $2^{n/2}$. Le dépassement de la ligne de capacité des ordinateurs personnels (fixée ici à 16 Go)

illustre de manière spectaculaire la frontière entre la validité mathématique d'un algorithme et son applicabilité dans le monde de la programmation logicielle et matérielle.

Cette limitation physique absolue justifie la nécessité de recourir à des variantes de l'algorithme plus économes en ressources. Pour contourner ce « mur de la RAM », les auteurs explorent deux voies successives : une première approche radicale supprimant totalement le stockage, suivie d'une approche plus équilibrée reposant sur un compromis temps/mémoire.

5 L'algorithme sans mémoire (*memoryless*) : le prix du temps

Pour pallier le problème rédhibitoire de l'empreinte spatiale de l'algorithme fondamental, les auteurs présentent à la page 13 de leur article une variante dite « sans mémoire » (*memoryless*). Cette approche permet de ramener la consommation de mémoire vive à une constante $\mathcal{O}(1)$.

Cette variante abandonne complètement la génération d'un large échantillon stocké dans une table de hachage. À la place, elle utilise l'algorithme de détection de cycles de Pollard pour trouver des collisions dans la fonction g_γ à la volée, une par une. Chaque fois qu'une collision est détectée, la différence (α, β) correspondante est immédiatement extraite et soumise à la phase de vérification exhaustive. L'algorithme ne stocke aucun historique.

Cependant, cette économie de mémoire se traduit par une pénalité significative sur la complexité en temps. Puisque l'algorithme perd la vision globale offerte par le paradoxe des anniversaires sur un grand tableau, il doit évaluer de très nombreuses collisions inutiles avant de tomber sur un quadruplet correct. Plus précisément, il faut extraire en moyenne $\mathcal{O}(p^{-2})$ collisions de la fonction g_γ pour obtenir une correspondance valide. La recherche d'une collision par la méthode de Pollard coûtant $\mathcal{O}(2^{n/2})$, la complexité temporelle globale grimpe à :

$$\text{Temps}_{\text{memoryless}} = \mathcal{O}(2^{n/2}) \times \mathcal{O}(p^{-2}) = \mathcal{O}(2^{n/2}p^{-2})$$

(ceci étant valable dans le cas où $p \geq 2^{-n/2}$).

Si l'on compare cette complexité à celle de l'algorithme fondamental, le rapport des temps d'exécution met en évidence l'ampleur du ralentissement :

$$\frac{\text{Temps}_{\text{memoryless}}}{\text{Temps}_{\text{fondamental}}} = \frac{2^{n/2}p^{-2}}{2^{n/2}p^{-1}} = p^{-1}$$

L'approche sans mémoire subit donc un **ralentissement direct d'un facteur p^{-1}** .

Pour bien mesurer l'impact de ce facteur, prenons un exemple concret sur le chiffrement Speck 32/64 ($n = 32$) avec une différentielle de probabilité $p = 2^{-13}$. Le calcul de la pénalité temporelle donne :

$$p^{-1} = (2^{-13})^{-1} = 2^{13} = 8192$$

Concrètement, cela signifie qu'une recherche qui prendrait **1 minute** avec l'algorithme fondamental prendrait 8192 fois plus de temps avec cette variante sans mémoire. En détaillant la conversion :

$$8192 \text{ minutes} = \frac{8192}{60} \text{ heures} \approx 136,5 \text{ heures} \approx \mathbf{5,68 \text{ jours}}$$

De plus, plus la probabilité p recherchée est faible, plus le temps d'exécution augmente. Bien que hautement parallélisable, cette méthode s'avère donc beaucoup trop lente pour être exploitée efficacement sur des machines standards, qui disposent pourtant de plusieurs gigaoctets de RAM laissés inutilisés par cette approche. C'est précisément pour combler ce vide entre le « tout mémoire » (incalculable) et le « sans mémoire » (interminable) que la solution du compromis a été développée.

6 L'algorithme de compromis temps/mémoire (tradeoff)

Pour surmonter ce dilemme, les auteurs proposent en annexe de leur article d'exploiter la mémoire disponible (notée S) de façon stratégique pour concevoir des algorithmes de compromis (*Tradeoffs*). Ils décrivent plus précisément deux variantes distinctes, adaptées à différentes capacités matérielles :

- **Le Premier Compromis (Faible mémoire) :** Il s'agit d'une extension de l'algorithme dit « sans mémoire ». Conçu pour des environnements où l'espace S est extrêmement restreint, il trouve un petit nombre de collisions et les teste toutes de manière systématique. Cette phase de vérification exhaustive alourdit considérablement la complexité temporelle (ajoutant un terme en $\tilde{O}(p^{-3})$), le rendant sous-optimal sur des machines classiques.
- **Le Second Compromis (Mémoire modérée) :** Il s'agit d'une extension de l'algorithme fondamental. Conçu pour des valeurs de S plus larges, il élimine la coûteuse phase de test systématique en cherchant directement des collisions multiples pour filtrer le bruit en amont.

Dans le cadre de ce projet, les ordinateurs modernes standards disposant généralement d'une quantité de mémoire vive confortable (de 8 à 16 Go de RAM), nous ne sommes pas dans un scénario de mémoire critique absolue. Nous nous sommes donc naturellement penchés sur le **Second Algorithme de Compromis** (*Tradeoff algorithm 2*). Cette approche permet de conserver la vitesse d'exécution optimale en $\tilde{O}(2^{n/2}p^{-1})$ tout en plafonnant intelligemment l'utilisation de la mémoire à un espace fixe dimensionné à $\tilde{O}(p^{-2})$.

6.1 Principe théorique : l'approche probabiliste par lots (batching)

Pour concevoir ce second compromis, les auteurs de l'article s'appuient sur l'algorithme de recherche de collisions parallèles (PCS), introduit par van Oorschot et Wiener en 1999 [2]. L'étude détaillée de la mécanique interne de cet algorithme dépasse le cadre de notre implémentation ; nous renvoyons le lecteur à l'article original pour une description exhaustive.

D'un point de vue fonctionnel, il suffit de retenir que la méthode PCS permet d'extraire un lot de S collisions de la fonction g_γ tout en maintenant une empreinte spatiale strictement bornée à $\mathcal{O}(S)$. Le temps d'exécution requis pour générer ce lot unique s'élève à $\mathcal{O}(2^{n/2} \cdot S^{1/2})$.

La question centrale devient alors de déterminer combien de lots successifs de taille S doivent être générés pour détecter les différentielles à haute probabilité ciblées. Pour y répondre, il faut analyser les probabilités au sein d'un seul lot et évaluer ses chances de succès.

Une vraie différentielle produit un « quadruplet correct » (une collision utile) avec une probabilité de p^2 . Dans un lot contenant S collisions quelconques, l'espérance de trouver une collision liée à l'une de ces différentielles réelles est donc de $S \cdot p^2$. Cependant, pour éviter les faux positifs, l'algorithme exige de trouver au moins deux collisions au sein du même lot suggérant exactement la même différence d'entrée-sortie (α, β) . En probabilités, lorsque l'espérance d'un événement rare est $\lambda = S \cdot p^2$, la probabilité que cet événement se produise au moins deux fois est proportionnelle à λ^2 . La probabilité de succès pour un lot entier est donc d'environ :

$$P(\text{succès d'un lot}) \approx (S \cdot p^2)^2 = S^2 \cdot p^4$$

Puisque la probabilité d'obtenir ce résultat avec un seul lot est de $S^2 \cdot p^4$, il faudra, en moyenne, répéter la recherche avec de nouvelles saveurs γ un nombre de fois équivalent à l'inverse de cette probabilité. Le nombre de lots nécessaires est donc de $S^{-2} \cdot p^{-4}$.

La complexité temporelle globale théorique s'obtient en multipliant le temps passé à générer un lot par le nombre de lots itérés :

$$\text{Temps total} = \underbrace{\tilde{O}(2^{n/2}S^{1/2})}_{\text{Temps pour 1 lot}} \times \underbrace{S^{-2}p^{-4}}_{\text{Nombre de lots}} = \tilde{O}(2^{n/2}p^{-4}S^{-3/2})$$

L'intérêt de cette formule apparaît lorsque l'on fixe la limite de mémoire allouée à $S = \tilde{O}(p^{-2})$. En injectant cette valeur dans l'équation de la complexité temporelle, les exposants s'annulent :

$$\text{Temps total} = \tilde{O}\left(2^{n/2}p^{-4}(p^{-2})^{-3/2}\right) = \tilde{O}\left(2^{n/2}p^{-4}p^3\right) = \tilde{O}(2^{n/2}p^{-1})$$

Ainsi, l'algorithme théorique retrouve la complexité temporelle optimale de l'approche fondamentale en $\tilde{O}(2^{n/2}p^{-1})$, tout en limitant l'empreinte spatiale à $\mathcal{O}(p^{-2})$.

6.2 De la théorie à la pratique : implémentation par lots et accumulation globale

Bien que l'algorithme PCS soit optimal d'un point de vue asymptotique, il a été originellement conçu pour des environnements où la contrainte mémoire est extrême. Son implémentation pratique s'avère complexe : elle exige un paramétrage dynamique des points distingués, une gestion précise des cycles infinis, et le recalcul des chaînes ajoute une complexité en temps non négligeable.

Dans le cadre de ce projet, nos machines de test disposaient d'une mémoire vive modérée (plusieurs gigaoctets). Il n'était donc ni nécessaire ni pertinent de s'imposer la rigueur spatiale absolue du PCS. Nous avons fait le choix assumé d'allouer une empreinte mémoire légèrement supérieure pour bénéficier d'une architecture beaucoup plus simple et rapide en C++ (fonction `runMemoryEfficientAlgorithm`). Plutôt que d'utiliser les chaînes de points distingués, nous avons implémenté une **recherche par lots restreints (batching) couplée à un tri en mémoire**.

Le réglage empirique de la RAM (M_{MAX}) : Le premier défi fut de calibrer la limite stricte de RAM autorisée. Lors de nos tests, nous avons constaté qu'au-delà d'une allocation de 2,3 Go (soit environ 100 millions d'entrées), le système d'exploitation commençait à utiliser la mémoire virtuelle sur le disque dur (*swap*). Ce mécanisme ralentissait l'exécution de manière si drastique que le gain théorique de l'algorithme était annulé. L'algorithme plafonne donc dynamiquement la taille d'un lot à cette valeur M_{MAX} .

L'évaluation et le tri des lots : Pour un substitut γ donné, le programme remplit le tableau alloué avec des évaluations aléatoires de g_γ . Au lieu d'utiliser une structure de hachage, le tableau est trié en place. Par conséquent, toutes les collisions se retrouvent mécaniquement adjacentes. Un unique parcours linéaire du tableau permet alors de les identifier et d'en extraire les candidats. Cette approche s'avère très économe en espace et extrêmement rapide, car le tri en mémoire contiguë maximise l'utilisation du cache du processeur.

La compensation mathématique (l'effet K^2) : D'après le paradoxe des anniversaires, le nombre de paires formées évolue de manière quadratique. Diviser la taille du tableau idéal d'un facteur K (pour respecter M_{MAX}) fait donc chuter le nombre de collisions générées par lot d'un facteur K^2 . Pour compenser cette perte statistique et conserver le même taux de détection global, le programme tire une nouvelle saveur γ aléatoire à chaque itération et exécute rigoureusement K^2 lots indépendants. Plus la mémoire allouée est faible, plus le temps d'exécution augmente quadratiquement.

Le filtre de fréquence global : La fragmentation de la recherche impose de compter les apparitions d'un candidat (α, β) à travers les différents lots, sans recréer un goulot d'étranglement mémoire. Notre implémentation utilise un **filtre de fréquence** (*Frequency Filter*) pré-alloué de taille fixe (1 Go). Lorsqu'une collision suggère une clé, celle-ci est hachée et indexée dans ce grand tableau d'octets pour incrémenter un compteur. Ce n'est que si cette case franchit le seuil d'acceptation fondamental ($n/4$) que la véritable clé est allouée dans une liste restreinte pour être soumise à la vérification finale.

Validation expérimentale : Pour démontrer l'impact de ce compromis, nous avons mené une campagne de tests sur Speck 32/64 ($p = 2^{-9}$), dont l'échantillon idéal nécessite environ 5,6 Go de RAM. Nous avons artificiellement restreint la limite mémoire allouée à notre algorithme pour observer son comportement (Figure 4).

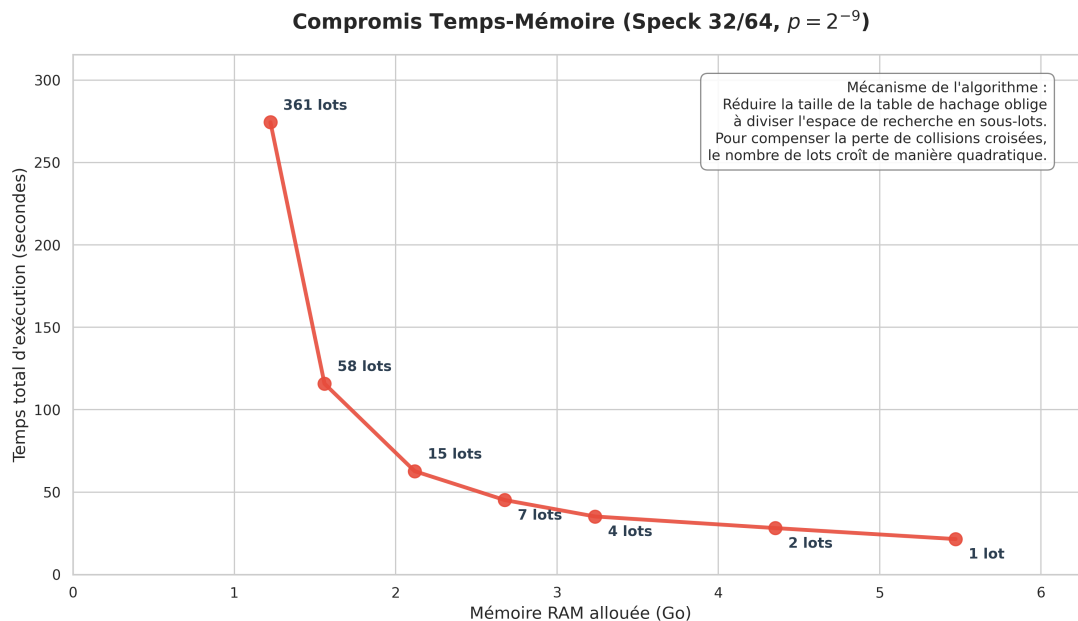


Figure 4 – Courbe du compromis Temps-Mémoire sur Speck 32/64. La réduction de la RAM disponible contraint l'algorithme à multiplier les lots d'évaluation, entraînant une hausse quadratique du temps d'exécution (effet K^2).

Les résultats empiriques confirment parfaitement le modèle mathématique. En réduisant l'empreinte mémoire d'un facteur 4,5 (passant de 5,6 Go à 1,25 Go), le temps d'exécution passe de 21 secondes à plus de 274 secondes, et le nombre de lots nécessaires augmente de 1 à 361. L'algorithme de compromis offre ainsi une adaptabilité matérielle au prix d'un allongement prévisible du calcul.

7 Ouverture : La dépendance aux hypothèses

Grâce à cet algorithme de compromis, la détection de différentielles à haute probabilité devient réalisable sur du matériel standard, franchissant définitivement la barrière de la recherche exhaustive en 2^n .

Cependant, il est crucial de noter qu'aussi bien l'algorithme fondamental que ses variantes (sans mémoire ou de compromis) reposent sur une hypothèse fondatrice : le comportement pseudo-aléatoire de la fonction substitut g_γ . En effet, l'évaluation des probabilités, le nombre de quadruplets attendus et la répartition des collisions supposent que la primitive analysée ne présente pas de structures internes venant fausser ces statistiques.

Si le chiffrement étudié possède des propriétés algébriques particulières qui empêchent g_γ de se comporter comme une fonction aléatoire, les garanties mathématiques de détection s'effondrent. Cette dépendance aux hypothèses justifie la nécessité de concevoir un dernier filet de sécurité : un algorithme « pire cas » (*Worst-Case Algorithm*). Bien que plus lourd, ce dernier doit être capable de retrouver ces propriétés statistiques sans formuler la moindre hypothèse sur le comportement structural de la primitive étudiée.

8 L'algorithme « pire cas » (*worst-case*) : s'affranchir des heuristiques

Comme évoqué précédemment, les algorithmes fondamental et de compromis reposent sur un pari fort : l'hypothèse que les quadruplets corrects se distribuent de manière aléatoire au sein de l'espace des messages. Cependant, si un concepteur malveillant introduit une porte dérobée (*trapdoor*) dans le chiffrement, par exemple, en forçant les paires correctes à se concentrer dans un sous-espace linéaire restreint, l'algorithme fondamental échouera presque systématiquement, car il n'aura testé qu'un ou très peu de substituts γ .

Pour contrer cette menace et garantir une détection infaillible même face à une primitive conçue de manière adverse, les auteurs de l'article introduisent un algorithme dit « pire cas ». Son principe est de ne plus s'en remettre à la chance sur le choix du substitut, mais de forcer statistiquement la découverte en multipliant les angles d'attaque de manière indépendante.

8.1 Paramétrage et seuils de validation

Au lieu d'utiliser un échantillon géant avec un seul substitut γ (ou de chercher jusqu'à trouver un succès comme dans le tradeoff), l'algorithme *Worst-Case* va tirer un nombre prédéfini et massif de substituts γ différents, et générer un sous-échantillon pour chacun d'eux. La mécanique repose sur trois paramètres mathématiques interconnectés, rigoureusement calibrés pour isoler le signal du bruit sans formuler d'hypothèses sur la distribution.

1. La taille du sous-échantillon M : La première différence majeure avec l'algorithme fondamental réside dans le dimensionnement de M . L'algorithme fondamental cherchait à accumuler suffisamment de collisions globales en une seule fois, exigeant un M colossal. Ici, l'objectif d'un seul lot est beaucoup plus modeste : pour un substitut γ donné, le lot doit être juste assez grand pour garantir la présence d'**au moins un** quadruplet correct (ce qui constituera un « vote » pour ce γ).

Pour garantir une forte probabilité de succès (fixée par les auteurs à 80 %, soit $4/5$), ces derniers s'appuient sur la méthode probabiliste du second moment dans la démonstration de leur Lemme 1. Cette méthode indique que pour atteindre le seuil de $4/5$, l'espérance mathématique du nombre de quadruplets corrects doit être d'au moins 8. Sachant que cette espérance se calcule par $E = \frac{M^2}{2} \cdot 2^{-n} \cdot p$, on peut en déduire la taille optimale du lot :

$$\frac{M^2}{2} \cdot 2^{-n} \cdot p = 8 \implies M^2 = 16 \cdot 2^n \cdot p^{-1} \implies M = 4 \cdot 2^{n/2} \cdot p^{-1/2}$$

C'est pourquoi M évolue ici en $p^{-1/2}$ et non plus en p^{-1} , rendant chaque lot beaucoup plus léger à stocker en mémoire.

2. Le seuil d'acceptation global ($0,28 \cdot S \cdot p$) : À la fin du traitement d'un lot, si une différence (α, β) a été repérée, le substitut γ donne « un vote » pour ce candidat (un seul vote maximum par γ , pour éviter qu'un sous-espace dense ne fausse le comptage). Le programme doit ensuite décider combien de votes sont nécessaires sur l'ensemble des itérations pour valider une différentielle.

Ce seuil est obtenu en calculant l'espérance des votes d'un vrai signal par rapport à un bruit statistique :

- **Pour une vraie différentielle (probabilité $\geq p$) :** Comme établi par la taille de M , le lot a déjà une probabilité de $4/5$ de contenir une bonne paire initiale. La question est de savoir si la paire translatée par le substitut aléatoire γ sera elle aussi correcte. Dans un scénario idéal, cette probabilité conditionnelle serait de p . Toutefois, pour garantir que l'algorithme fonctionne même dans le pire des cas (où un adversaire aurait biaisé la distribution), les auteurs démontrent mathématiquement que cette probabilité ne peut jamais descendre sous la borne stricte de $p/2$. La probabilité minimale qu'un lot génère un vote est donc le

produit de ces deux étapes : $(4/5) \times (p/2) = 2p/5 = 0,40 \cdot p$. Sur S itérations, on s'attend à récolter en moyenne $0,40 \cdot S \cdot p$ votes.

- **Pour une fausse différentielle (probabilité $\leq p/10$)** : Ici, aucune paire n'est garantie par la taille du lot. La probabilité qu'une fausse piste forme un quadruplet par pur hasard nécessite de valider les deux paires consécutivement, ce qui force à élever sa probabilité au carré. L'évaluation de l'espérance (qui dépend de M^2 , introduisant ainsi un facteur 16 puisque M commence par 4) donne un plafond de $16 \cdot p \cdot (1/10)^2 = 0,16 \cdot p$. L'espérance maximale des votes pour un bruit est donc de $0,16 \cdot S \cdot p$.

Pour séparer le signal du bruit, le seuil d'acceptation est fixé exactement à la moyenne arithmétique entre ces deux espérances (0,40 et 0,16) :

$$\text{Seuil} = \frac{0,40 + 0,16}{2} \cdot S \cdot p = 0,28 \cdot S \cdot p$$

3. Le nombre d'itérations S : Fixer une moyenne parfaite ne suffit pas : il faut répéter l'expérience un nombre de fois S suffisamment grand pour se prémunir des fluctuations statistiques. Sachant qu'il existe 2^{2n} combinaisons possibles de fausses différentielles (α, β) , la probabilité qu'une seule d'entre elles franchisse le seuil par erreur doit être infime (strictement inférieure à 2^{-2n}).

Dans le Lemme 2 de l'article, les auteurs utilisent la **borne de Chernoff** pour évaluer cette probabilité d'erreur, qui s'exprime sous la forme exponentielle $e^{-\frac{S \cdot p}{50}}$. Afin de sécuriser le résultat face aux 2^{2n} fausses pistes, ils choisissent d'imposer un taux d'erreur encore plus strict, fixé à e^{-4n} (soit environ $2^{-5,7n}$). Pour trouver le nombre d'itérations S nécessaire, il suffit de résoudre l'égalité entre ces deux exposants :

$$\frac{S \cdot p}{50} = 4n \implies S \cdot p = 200n \implies S = \frac{200n}{p}$$

Le paramètre constant 200 découle directement de cette contrainte de sécurité. En remplaçant S dans notre équation de seuil, on obtient finalement la condition de validation implémentée dans notre code : un candidat est validé s'il obtient plus de $0,28 \cdot \left(\frac{200n}{p}\right) \cdot p = 56n$ votes.

8.2 Complexités théoriques et implémentation

Cette robustesse absolue a un coût. La complexité temporelle globale de l'algorithme s'obtient en multipliant le nombre d'itérations S par la taille du lot M à évaluer à chaque fois :

$$\text{Temps} = \tilde{O}(S \cdot M) = \tilde{O}\left(p^{-1} \times 2^{n/2} p^{-1/2}\right) = \tilde{O}(2^{n/2} p^{-3/2})$$

Par rapport à l'algorithme fondamental $\tilde{O}(2^{n/2} p^{-1})$, cette approche pire cas subit un ralentissement d'un facteur $\tilde{O}(p^{-1/2})$. En revanche, sa complexité spatiale est limitée au stockage de la table de hachage d'un seul lot M à la fois, soit $\tilde{O}(2^{n/2} p^{-1/2})$, ce qui représente un excellent compromis.

Notre implémentation C++ de cette méthode, encapsulée dans la fonction `runWorstCaseAlgorithm`, exploite pleinement la nature indépendante des itérations S . La boucle principale gérant les différents substituts γ a été parallélisée. Chaque thread alloue sa propre table de hachage temporaire pour analyser un sous-échantillon M . Il ne vient mettre à jour le grand compteur global des votes qu'à la fin de son évaluation. Une fois les S itérations terminées, le programme applique le filtre final des $56n$ votes et exporte les différentielles certifiées.

8.3 Limites pratiques : le cumul des contraintes

Bien que cette variante offre des garanties théoriques absolues de sécurité, elle cumule en pratique les défauts majeurs des deux algorithmes précédents : une forte consommation de

RAM (héritée de l'algorithme fondamental) et une grande lenteur (héritée de l'approche sans mémoire).

En effet, le ralentissement théorique d'un facteur $p^{-1/2}$ et la nécessité de stocker M messages en mémoire simultanément rendent l'exécution extrêmement lourde. Pour s'en rendre compte, reprenons notre exemple numérique sur le chiffrement Speck 32/64 ($n = 32$) avec une probabilité $p = 2^{-13}$:

- **L'obstacle temporel** : Le facteur de ralentissement $p^{-1/2}$ vaut ici $\sqrt{2^{13}} \approx 90,5$. Cela signifie qu'une recherche qui prenait **1 minute** avec l'algorithme fondamental prendra ici **plus d'une heure et demie**.
- **L'obstacle spatial (RAM)** : Pour ces paramètres, la taille d'un sous-échantillon M atteint environ 23,7 millions de messages. En pratique, une table de hachage pèse environ 32 octets par entrée en raison des métadonnées requises par la structure de données sous-jacente (gestion des collisions, pointeurs internes). Chaque fil d'exécution consomme donc près de 750 Mo. Sur un processeur standard exécutant 8 fils en parallèle, l'algorithme accapare instantanément **6 Go de mémoire vive** pour analyser un simple chiffrement « jouet » de 32 bits.

Une piste d'amélioration théorique (non implémentée dans nos travaux) consisterait à appliquer la méthode de recherche de collisions parallèles (PCS) au sein même de cet algorithme « pire cas », à l'instar de ce qui a été fait pour le compromis temps/mémoire. Cela permettrait d'éliminer le stockage direct des M messages dans des tables de hachage et réduirait drastiquement l'empreinte spatiale. Néanmoins, bien que le problème de la RAM puisse être atténué par cette astuce, la barrière temporelle imposée par la complexité en $\tilde{O}(2^{n/2}p^{-3/2})$ resterait un obstacle infranchissable pour analyser des chiffrements avec des tailles de blocs standards ($n \geq 64$).

8.4 Bilan et synthèse comparative des algorithmes

Au terme de cette analyse détaillant la transition de la recherche exhaustive vers la recherche probabiliste, il convient de mettre en perspective l'ensemble des méthodes explorées. Le Tableau 1 dresse le bilan des six variantes algorithmiques abordées dans ce rapport. Il confronte leurs complexités en temps et en espace ainsi que leurs dépendances respectives aux hypothèses de la primitive étudiée.

Table 1 – Comparaison théorique des quatre variantes algorithmiques étudiées.

Algorithme	Complexité temps	Complexité espace	Hypothèses / Contraintes
Naïf exhaustif	$\mathcal{O}(2^{2n})$	$\mathcal{O}(2^{2n})$	Aucune. DDT complète. Inutilisable dès $n > 32$.
Naïf optimisé (pDDT)	$\mathcal{O}(2^n \cdot p^{-1})$	$\mathcal{O}(2^n)$	Aucune.
Fondamental (collisions)	$\tilde{\mathcal{O}}(2^{n/2} \cdot p^{-1})$	$\tilde{\mathcal{O}}(2^{n/2} \cdot p^{-1})$	g_γ pseudo-aléatoire. Mémoire prohibitive dès $n \geq 32$.
Sans mémoire	$\tilde{\mathcal{O}}(2^{n/2} \cdot p^{-2})$	$\mathcal{O}(1)$	g_γ pseudo-aléatoire.
Compromis	$\tilde{\mathcal{O}}(2^{n/2} \cdot p^{-1})$	$\tilde{\mathcal{O}}(p^{-2})$	g_γ pseudo-aléatoire. Plafond mémoire $S = \tilde{\mathcal{O}}(p^{-2})$.
Pire cas	$\tilde{\mathcal{O}}(2^{n/2} \cdot p^{-3/2})$	$\tilde{\mathcal{O}}(2^{n/2} \cdot p^{-1/2})$	Aucune. Robuste face à toute distribution adverse.

9 Analyse comparative des performances sur SPN 16 bits

Afin d'évaluer l'efficacité des différentes stratégies de recherche, nous avons mené une campagne de tests sur un chiffrement SPN de 16 bits (3 tours, $p = 2^{-7}$). Les évaluations ont été exécutées de manière strictement séquentielle sur un échantillon fixe de 500 clés aléatoires pour lisser les éventuelles variations matérielles et garantir la fiabilité des moyennes temporelles.

Par ailleurs, l'algorithme de « pire cas » a dû être exclu de ce comparatif. La garantie de sécurité mathématique qu'il offre impose un nombre d'itérations prohibitif ($S = 200n/p$), entraînant une surcharge temporelle et spatiale qui rend son exécution impraticable, même sur un bloc réduit de 16 bits. Les résultats des méthodes évaluées sont synthétisés dans le Tableau 2.

Table 2 – Temps moyens d'exécution des algorithmes sur SPN 16 bits (moyenne sur 500 clés).

Algorithme	Temps moyen (secondes)
Naïf Exhaustif	12,239
Naïf Optimisé	3,373
Compromis Temps-Mémoire	2,984
Fondamental	2,831

L'analyse de cette hiérarchie de performances confirme la pertinence des modèles théoriques. L'algorithme naïf exhaustif s'effondre logiquement avec un temps moyen de 12,239 secondes, illustrant l'impossibilité pratique d'une recherche par force brute à plus grande échelle. Si l'approche naïve optimisée divise ce temps par près de quatre (3,373 s), elle reste fondamentalement limitée par son exploration séquentielle en $\mathcal{O}(2^n)$.

Les approches exploitant les collisions révèlent quant à elles pleinement leur supériorité. L'algorithme fondamental (2,831 s) s'impose comme la solution la plus performante, validant empiriquement l'avantage de la complexité en $\mathcal{O}(2^{n/2} \cdot p^{-1})$ permise par le paradoxe des anniversaires.

On observe logiquement que la variante de compromis temps-mémoire (2,984 s) se place très légèrement en retrait. Cet écart s'explique par la surcharge logicielle (*overhead*) induite par le découpage en lots successifs et la gestion continue des filtres. Sur une primitive de 16 bits où la quantité de RAM n'est pas un facteur limitant, l'algorithme fondamental pur demeure donc l'option la plus directe et la plus rapide.

10 Les limites de l'approche naïve face à Speck 32/64 (3 tours)

Pour illustrer concrètement l'explosion combinatoire liée à l'augmentation de la taille du bloc, nous avons transposé nos expérimentations sur le chiffrement Speck 32/64. Afin de conserver des temps de calcul mesurables pour l'algorithme naïf optimisé, nous avons ciblé une caractéristique sur un nombre réduit de 3 tours, offrant une probabilité théorique très favorable de $p = 2^{-3}$.

Le Tableau 3 synthétise les résultats. Contrairement aux tests précédents sur 16 bits, les limites matérielles et temporelles nous ont obligés à repenser notre protocole.

Table 3 – Performances de recherche différentielle (Cible : Speck32/64, 3 tours, $p = 2^{-3}$). L'algorithme naïf exhaustif a été interrompu, son temps de calcul étant estimé à plusieurs années.

Algorithme	Complexité	Moyenne (s)	Écart-type	Accélération
Naïf Exhaustif	$\mathcal{O}(2^{2n})$	-	-	-
Naïf Optimisé	$\mathcal{O}(2^n \cdot p^{-1})$	10 503,31	0,00	Référence (1×)
Fondamental	$\mathcal{O}(2^{n/2} \cdot p^{-1})$	1,92	0,04	$\sim 5447\times$

L'analyse de ces résultats met en évidence le « mur de la scalabilité » auquel se heurtent les approches triviales. L'attaque par force brute pure ($\mathcal{O}(2^{2n})$) exige l'évaluation de 2^{64} paires, un volume de calcul prohibitif qui justifie son abandon et prouve son inapplicabilité dès $n = 32$. De son côté, l'algorithme naïf optimisé atteint également ses limites : il requiert près de trois heures (10 500 s) pour traiter une attaque sur trois tours ($p = 2^{-3}$). Cibler des différentielles plus profondes (ex. $p = 2^{-9}$ sur cinq tours) exigerait dès lors des mois de calcul ininterrompu.

À l'inverse, l'algorithme fondamental démontre la supériorité de l'approche par collisions ($\mathcal{O}(2^{n/2} \cdot p^{-1})$). En résolvant le même problème en moins de deux secondes, il offre une accélération massive d'un facteur 5447 par rapport à la méthode naïve optimisée. Ce résultat illustre le point de bascule théorique permis par l'exploitation du paradoxe des anniversaires.

Ce comparatif grandeur nature confirme les observations faites sur le SPN 16 bits. Ce qui n'apparaissait que comme un léger avantage temporel sur un petit bloc se transforme en un gouffre de performance infranchissable sur une primitive de 32 bits.

11 Validation expérimentale et résolution matérielle (Speck 5 et 6 tours)

11.1 Reproduction des résultats sur 5 tours de Speck 32/64

Pour valider notre implémentation, nous avons reproduit les résultats de l'article de référence (Section 2.5) en étudiant la meilleure caractéristique différentielle sur 5 tours de Speck 32/64 connue à ce jour [3] :

$$(0x0211, 0x0a04) \rightarrow (0x8000, 0x840a)$$

La probabilité théorique attendue pour cette différentielle est d'environ $p \approx 2^{-9}$.

Atteindre ce seuil avec l'algorithme fondamental nécessiterait de stocker environ 189 millions de messages, ce qui saturerait instantanément la mémoire de nos machines. Pour contourner ce problème, nous avons utilisé l'algorithme de compromis Temps/Mémoire. La recherche a été divisée en lots limités à 100 millions de textes clairs (soit environ 2,2 Go de RAM). Pour vérifier si ce résultat dépendait de la clé, le programme a tourné sur un échantillon de 1000 clés générées aléatoirement, ce qui a pris environ 30 heures de calcul.

Les résultats, présentés sur la Figure 5, montrent un comportement inattendu. D'un côté, pour les 608 clés qui ont dépassé le seuil de détection, la moyenne mesurée est de $\mu \approx 2^{-8,98}$, ce qui confirme très bien la probabilité de 2^{-9} donnée par la littérature. De l'autre côté, l'algorithme n'a trouvé aucune collision pour les 392 clés restantes, affichant une probabilité de 0.

Face à ce grand nombre de faux négatifs (39, 2 % de l'échantillon), nous avons cherché à savoir si ce problème venait du chiffrement lui-même (certaines clés bloquant la différentielle) ou s'il s'agissait d'un défaut de notre outil de recherche.

Pour le vérifier, nous avons pris un groupe de contrôle de 100 clés non détectées ($p = 0$) et de 100 clés bien détectées ($p > 0$). Nous les avons testées directement : pour chaque clé, 2^{20} paires aléatoires ont été générées et chiffrées pour estimer la probabilité exacte, sans utiliser la méthode des collisions et ses filtres. Les résultats de ce test sont clairs : les 200 clés affichent toutes une probabilité proche de 2^{-9} . La différentielle existe donc bien pour ces 200 clés.

Le fait que notre algorithme par compromis rate ces 392 clés souligne les limites de cette méthode. Plusieurs raisons peuvent expliquer ces échecs.

La première explication réside dans les limites de l'hypothèse d'indépendance. L'algorithme suppose en effet que la fonction g_γ se comporte de manière aléatoire. Puisque l'algorithme de compromis limite la recherche à quelques lots, il n'utilise qu'un nombre très restreint de substituts γ différents. Il suffit alors que ces quelques substituts s'accordent mal avec la clé testée pour qu'aucune collision ne se forme, conduisant l'algorithme à passer complètement à côté du résultat.

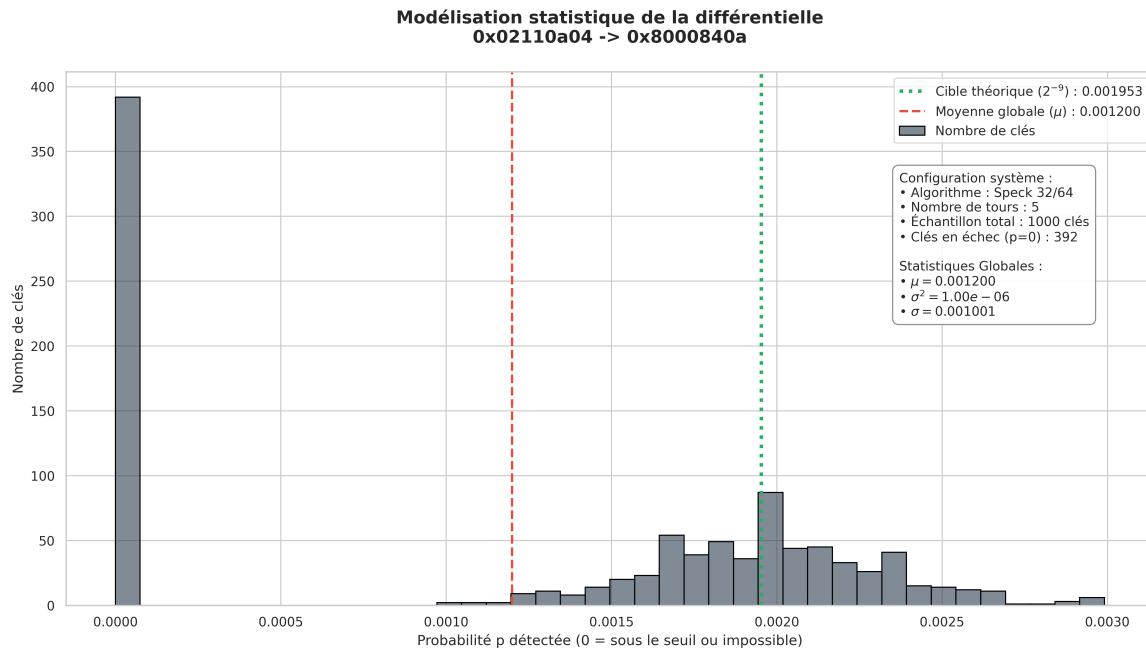


Figure 5 – Distribution globale de la probabilité différentielle sur 1000 clés (Speck 32/64, 5 tours). Le graphique met en évidence un nombre important de faux négatifs (392 clés non détectées) contrastant avec la moyenne des clés validées, centrée sur $p \approx 2^{-9}$.

À cette contrainte théorique s'ajoute un manque de détails sur l'implémentation pratique. L'algorithme de compromis n'est abordé que de façon conceptuelle dans les annexes des travaux de Dinur et al. Les auteurs indiquent qu'il faut générer des lots successifs de collisions en modifiant le substitut γ à chaque itération, mais la gestion concrète de la mémoire et le transfert des candidats d'un lot à l'autre ne sont pas formalisés en pseudo-code. Nos propres choix de programmation pour pallier ce manque ont donc pu introduire des biais de filtrage, éliminant par erreur de bons candidats. Pour s'affranchir de ces biais et garantir une détection certaine, il serait nécessaire d'utiliser l'algorithme de « pire cas », dont l'efficacité a été validée par les chercheurs de l'état de l'art grâce à l'évaluation massive de nombreux substituts γ . Cependant, comme nous l'avons établi précédemment, cette méthode exige des ressources matérielles beaucoup trop lourdes pour notre environnement de test.

Pour conclure, notre implémentation est parvenue à extraire la différentielle de manière entièrement automatisée pour 60,8 % des clés, alors même que notre vérification manuelle a prouvé que la différentielle était présente dans l'intégralité des cas. Ce résultat illustre directement la complexité liée à l'implémentation pratique de tels algorithmes. Le passage de la théorie mathématique à un outil logiciel contraint par la mémoire impose des choix d'architecture qui se heurtent parfois à la réalité algébrique du chiffrement. Cela confirme la pertinence et la nécessité de l'algorithme de « pire cas » pour assurer une vérification systématique et fiable, bien que son coût matériel reste en pratique prohibitif.

11.2 Reproduction des résultats sur 6 tours de Speck 32/64

Après avoir validé l'algorithme sur 5 tours, nous avons voulu reproduire le résultat de Dinur et al. sur **6 tours** de Speck 32/64. Avec 6 tours, la probabilité chute à $p \approx 2^{-13}$, ce qui allonge considérablement le temps de calcul.

À cause de cette très faible probabilité, l'algorithme fondamental pur aurait besoin de stocker plus de 3 milliards d'entrées en RAM ($M \approx 3 \times 10^9$). C'est impossible sur notre machine de test. C'est donc un cas d'usage adapté à l'utilisation de l'algorithme de compromis (*Tradeoff*).

Nous avons limité la RAM à 50 millions d'entrées par lot (environ 1,14 Go). Pour compenser ce manque d'espace, le programme a dû calculer **3690 lots** à la suite.

Table 4 – Les 6 meilleures différentielles trouvées sur Speck 32/64 (6 tours) avec l’algorithme Tradeoff. Le calcul a duré environ 6 heures avec 1,14 Go de RAM.

Différence Entrée (α)	Différence Sortie (β)	Probabilité mesurée	Conforme à l'article
0x02110a04	0x8d0a9d20	0,000145	Oui (Dinur <i>et al.</i>)
0x02110a04	0x850a9520	0,000092	
0x020f0a04	0x850a9520	0,000084	
0x02110a04	0x851a9530	0,000076	
0x02110a04	0x8f0a9f20	0,000076	
0x02110a04	0x830a9320	0,000072	

Après environ 6 heures de calcul (21 747 secondes), le programme a identifié 6 candidats (voir Tableau 4). Le premier candidat (0x8d0a9d20) affiche une probabilité mesurée de 0,000145, se plaçant ainsi devant le candidat (0x850a9520), évalué à 0,000092. Pourtant, ce dernier correspond à la meilleure différentielle connue dans la littérature pour 6 tours [3]. Afin de vérifier ce classement et de déterminer s’il s’agissait d’une véritable découverte ou d’un simple biais, nous avons procédé à une vérification globale sur 100 clés aléatoires, en testant 2^{24} paires de textes clairs par clé. En moyenne, la probabilité du premier candidat s’effondre à 0,000061, tandis que celle du candidat de la littérature se stabilise à 0,000114, confirmant ainsi son statut de meilleur candidat théorique ($2^{-13} \approx 0,000122$). Ce test sur 6 tours valide notre implémentation C++ et montre sa capacité à traiter de grands volumes de données tout en respectant les limites de la mémoire. Il rappelle également qu’une phase de vérification globale reste indispensable pour distinguer les véritables failles cryptographiques des simples erreurs d’évaluation.

12 Conclusion générale et perspectives

La cryptanalyse différentielle, bien que théorisée depuis plus de trente ans, continue de se heurter à un défi mathématique et physique absolu : l’explosion combinatoire. Au travers de ce Travail d’Étude et de Recherche, notre objectif n’était pas seulement d’étudier la littérature scientifique, mais de confronter les théories de l’état de l’art à la réalité de la programmation logicielle et matérielle.

Le développement de notre bibliothèque C++ nous a permis d’explorer l’intégralité du spectre algorithmique décrit dans le papier [3]. Nous avons d’abord implémenté les méthodes de recherche par force brute, dites naïves. Bien que ces dernières se révèlent efficaces sur des « chiffrements jouets » (notamment grâce à une parallélisation triviale sur 16 bits), leur complexité en temps et en mémoire en $\mathcal{O}(2^{2n})$ et $\mathcal{O}(2^n \cdot p^{-1})$ les condamne définitivement face à l’augmentation de la taille du bloc. L’effondrement de l’algorithme naïf optimisé sur Speck 32/64, nécessitant près de 3 heures de calcul pour analyser une simple caractéristique sur 3 tours, illustre parfaitement le « mur de la scalabilité ».

Pour briser ce mur, l’adoption de l’approche fondamentale de l’article, basée sur la différentielle de substitut et le paradoxe des anniversaires, marque une avancée algorithmique majeure. En réduisant la complexité temporelle à l’ordre de $\mathcal{O}(2^{n/2} \cdot p^{-1})$, cet algorithme est parvenu à expédier la même analyse sur Speck en moins de 2 secondes, représentant une accélération d’un facteur 5447.

Cependant, nos expérimentations ont également mis en lumière la limite physique de cet algorithme : son empreinte mémoire. Détecter des différentielles avec des probabilités aussi petite que $p = 2^{-9}$ sur un bloc de 32 bits requiert déjà plusieurs gigaoctets de mémoire vive, rendant l’algorithme fondamental pur inexploitable pour des paramètres de sécurité modernes (64 ou 128 bits). L’implémentation de la variante de compromis temps-mémoire, s’appuyant sur l’échantillonnage par lots et les filtres de fréquence, s’est alors imposée comme la solution indispensable. En acceptant un surcoût temporel prévisible et maîtrisé pour contraindre l’espace RAM alloué, nous avons prouvé qu’il est possible de faire aboutir des recherches complexes sur

des ordinateurs personnels standards.

Perspectives d'évolution

Notre moteur de recherche différentielle pose des fondations solides, mais le passage aux chiffrements répondant aux standards actuels ($n = 64$ ou $n = 128$ bits) exige une refonte profonde de l'architecture logicielle.

La première évolution majeure concernerait le dépassement de notre technique d'indexation binaire actuelle, strictement plafonnée à la fusion de deux valeurs de 32 bits. Pour supporter l'analyse de chiffrements aux standards modernes (opérant sur des blocs de 64 ou 128 bits), il deviendrait indispensable de repenser la représentation en mémoire des candidats différentiels. Cette mise à l'échelle exigerait l'adoption de structures de données nativement plus larges, couplée à l'exploitation des capacités de calcul vectoriel offertes par les processeurs récents. Afin de ne pas pénaliser les performances d'indexation lors des millions d'accès mémoire, l'intégration de fonctions de hachage non cryptographiques de dernière génération, spécifiquement optimisées pour digérer ultra-rapidement ces larges empreintes binaires, s'imposerait comme un prérequis absolu.

Enfin, pour surmonter l'augmentation inévitable des temps de calcul liée à l'allongement du nombre de tours, la migration de la méthode de recherche de collisions parallèles (PCS) des processeurs multi-cœurs (CPU) vers des architectures massivement parallèles (GPU) via CUDA ou OpenCL constituerait une évolution majeure. En déportant les évaluations massives des substituts g_γ sur des milliers de cœurs graphiques, il deviendrait envisageable de repousser encore plus loin les limites de la cryptanalyse différentielle pratique.

Références

- [1] E. Biham & A. Shamir, *Differential Cryptanalysis of DES-like Cryptosystems*, Journal of Cryptology, vol. 4, n°1, pp. 3–72, 1991. <https://link.springer.com/article/10.1007/BF00630563>
- [2] P. C. van Oorschot & M. J. Wiener, *Parallel Collision Search with Cryptanalytic Applications*, Journal of Cryptology, vol. 12, n°1, pp. 1–28, 1999. <https://link.springer.com/article/10.1007/PL00003816>
- [3] I. Dinur, O. Dunkelman, N. Keller, E. Ronen & A. Shamir, *Efficient Detection of High Probability Statistical Properties of Cryptosystems via Surrogate Differentiation*, Eurocrypt 2023. <https://eprint.iacr.org/2023/288.pdf>
- [4] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks & L. Wingers, *The SIMON and SPECK Families of Lightweight Block Ciphers*, 2013. <https://eprint.iacr.org/2013/404.pdf>
- [5] A. François, B. Dupré & B. Guibert, *Moteur d'analyse différentielle en C++*, 2026. <https://github.com/Redak92/cryptanalyse-differentielle>